ГУАП

КАФЕДРА № 44

ОТЧЕТ
ЗАЩИЩЕН С ОЦЕНКОЙ
ПРЕПОДАВАТЕЛЬ

канд. техн. наук , доцент		А. Ю. Молчанов
должность, уч. степень, звание	подпись, дата	инициалы, фамилия

ОТЧЕТ О ЛАБОРАТОРНОЙ РАБОТЕ №1

РАБОТА С ТАБЛИЦЕЙ СИМВОЛОВ

по курсу: Системное программное обеспечение

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ ГР. №	4341в	02.10.2026	Р.С. Кулаков
		подпись, дата	инициалы, фамилия

Санкт-Петербург
2026

1 1. Краткое изложение цели работы

Целью лабораторной работы является изучение методов организации таблиц символов (идентификаторов), а также изучение преимуществ и недостатков различных способов разрешения коллизий при использовании хеш-таблиц.

В ходе работы необходимо реализовать хранение множества идентификаторов в хеш-таблице, организованной в соответствии с заданной хеш-функцией и методом разрешения коллизий. Также необходимо реализовать поиск идентификаторов и определить среднее количество коллизий и сравнений при поиске.

2 Задание по лабораторной работе с описанием варианта

Необходимо разработать программу, которая получает набор идентификаторов из текстового файла и организует их хранение в таблице символов с использованием заданной хеш-функции и метода разрешения коллизий.

Программа должна обеспечивать:

- добавление идентификаторов в таблицу;
- поиск заданного идентификатора;
- вывод содержимого хеш-таблицы;
- подсчёт количества коллизий;
- определение среднего количества коллизий;
- определение количества сравнений при поиске идентификатора.

Согласно варианту №12 используется следующая комбинация методов:

- хеш-функция — сумма кодов первой и последней букв идентификатора;
- метод разрешения коллизий — метод цепочек.

Для получения индекса ячейки хеш-таблицы используется выражение:

h(A) = (code(first) + code(last)) mod M, где M — размер хеш-таблицы.

В программе используется размер таблицы M = 13.

Например, для идентификатора apple:

code('a') = 97,

code('e') = 101.

Следовательно:

h(apple) = (97 + 101) mod 13 = 3.

Таким образом, идентификатор apple помещается в ячейку с индексом 3

3 Схема организации хеш-таблицы

При использовании метода цепочек хеш-таблица состоит из двух частей:

1. массива buckets, содержащего индексы первых элементов цепочек;
2. основной таблицы entries, содержащей сами идентификаторы и ссылки на следующие элементы цепочки.

Пустая ячейка массива buckets обозначается значением -1.

Общая схема организации таблицы:

Хеш-таблица

bucket 0 ----> элемент 0 ----> элемент 7 ----> NULL
bucket 1 ----> NULL
bucket 2 ----> элемент 1 ----> элемент 4 ----> NULL
bucket 3 | ----> элемент 2 ----> NULL
…
bucket 12 ----> NULL

Основная таблица:

Index	Identifier	Next
0	identifier	7
1	identifier	-1
2	identifier	-1
3	identifier	5
…	…	…

При добавлении нового идентификатора сначала вычисляется его хеш. Если соответствующая ячейка хеш-таблицы свободна, в неё записывается индекс нового элемента основной таблицы.

Если ячейка уже занята, возникает коллизия. В этом случае новый идентификатор добавляется в конец соответствующей цепочки. Поле next предыдущего элемента изменяется так, чтобы оно указывало на новый элемент.

Таким образом, элементы с одинаковым значением хеш-функции объединяются в одну цепочку.

Метод цепочек позволяет хранить несколько идентификаторов, имеющих одинаковый хеш, без поиска дополнительных свободных ячеек хеш-таблицы.

4 Описание алгоритма поиска в хеш-таблице

Поиск идентификатора выполняется следующим образом.

1. Для искомого идентификатора вычисляется значение хеш-функции: h(A) = (code(first) + code(last)) mod M.
2. Полученное значение используется как индекс хеш-таблицы.
3. Из соответствующей ячейки buckets получается индекс первого элемента цепочки.
4. Если ячейка содержит значение -1, цепочка отсутствует, поэтому искомый идентификатор не найден.

5. Если цепочка существует, начинается последовательный просмотр её элементов.
6. Для каждого элемента выполняется сравнение его идентификатора с искомым.
7. Если идентификаторы совпадают, поиск завершается успешно.
8. Если идентификаторы не совпадают, по полю next выполняется переход к следующему элементу цепочки.
9. Если значение next равно -1, цепочка закончилась, а идентификатор не найден.

При поиске программа также подсчитывает количество выполненных сравнений. Например, если искомый идентификатор находится первым элементом цепочки, выполняется одно сравнение. Если он находится третьим элементом, выполняются три сравнения.

5 Листинг

```
#!/usr/bin/env kotlin

import java.io.File

// Элемент основной таблицы идентификаторов.
// next — индекс следующего элемента в цепочке.
// Значение -1 означает конец цепочки.
data class Entry(
    val name: String,
    var next: Int = -1
)

// Результат поиска.
data class SearchResult(
    val found: Boolean,
    val comparisons: Int
)

class HashTable(
```

```
    private val size: Int
) {

    // Хеш-таблица.
    // В каждой ячейке хранится индекс первого элемента цепочки.
    // -1 означает, что цепочка пуста.
    private val buckets = IntArray(size) { -1 }

    // Основная таблица идентификаторов.
    // Новые элементы добавляются в конец.
    private val entries = mutableListOf<Entry>()

    // Общее количество коллизий при вставке.
    private var totalCollisions = 0

    // Количество успешно добавленных идентификаторов.
    private var insertedCount = 0

    // Хеш-функция:
    // код первой буквы + код последней буквы.
    // Остаток от деления используется для получения
    // корректного индекса в таблице.
    private fun hash(name: String): Int {
        val firstCode = name.first().code
        val lastCode = name.last().code

        return (firstCode + lastCode) % size
    }

    // Вставка нового идентификатора.
    fun insert(name: String): Boolean {

        if (name.isEmpty()) {
            return false
        }

        // Проверяем, нет ли уже такого идентификатора.
        val existing = search(name)

        if (existing.found) {
            return false
        }
```

```
    val hashIndex = hash(name)

    // Если в ячейке уже есть элемент,
    // происходит коллизия.
    if (buckets[hashIndex] != -1) {
        totalCollisions++
    }

    // Индекс нового элемента в основной таблице.
    val newIndex = entries.size

    // Добавляем идентификатор.
    entries.add(Entry(name))

    // Если цепочка пустая,
    // новый элемент становится первым.
    if (buckets[hashIndex] == -1) {
        buckets[hashIndex] = newIndex
    } else {

        // Иначе идём по цепочке до последнего элемента.
        var currentIndex = buckets[hashIndex]

        while (entries[currentIndex].next != -1) {
            currentIndex = entries[currentIndex].next
        }

        // Добавляем новый элемент в конец цепочки.
        entries[currentIndex].next = newIndex
    }

    insertedCount++

    return true
}

// Поиск идентификатора.
fun search(name: String): SearchResult {

    if (name.isEmpty()) {
        return SearchResult(false, 0)
```

```
    }

    val hashIndex = hash(name)

    // Получаем первый элемент цепочки.
    var currentIndex = buckets[hashIndex]

    var comparisons = 0

    // Просматриваем цепочку.
    while (currentIndex != -1) {

        comparisons++

        if (entries[currentIndex].name == name) {
            return SearchResult(true, comparisons)
        }

        currentIndex = entries[currentIndex].next
    }

    return SearchResult(false, comparisons)
}

// Вывод содержимого таблицы.
fun printTable() {

    println()
    println("HASH TABLE")
    println("=".repeat(60))

    println("Hash index | Chain")
    println("-".repeat(60))

    for (i in buckets.indices) {

        print("%10d | ".format(i))

        var currentIndex = buckets[i]

        if (currentIndex == -1) {
            println("empty")
```

```
            continue
        }

        while (currentIndex != -1) {

            val entry = entries[currentIndex]

            print("[${currentIndex}: ${entry.name}]")

            if (entry.next != -1) {
                print(" -> ")
            }

            currentIndex = entry.next
        }

        println()
    }

    println("-".repeat(60))

    println()
    println("MAIN IDENTIFIER TABLE")
    println("=".repeat(60))

    println("Index | Identifier | Next")
    println("-".repeat(60))

    for (i in entries.indices) {
        println(
            "%5d | %-10s | %4d".format(
                i,
                entries[i].name,
                entries[i].next
            )
        )
    }

    println("-".repeat(60))
}

// Среднее количество коллизий.
```

```
    fun getAverageCollisions(): Double {
        if (insertedCount == 0) {
            return 0.0
        }

        return totalCollisions.toDouble() / insertedCount
    }

    // Получение количества коллизий.
    fun getTotalCollisions(): Int {
        return totalCollisions
    }

    // Получение количества элементов.
    fun getSize(): Int {
        return insertedCount
    }
}


// Размер хеш-таблицы.
val tableSize = 13

val hashTable = HashTable(tableSize)

val fileName = "identifiers.txt"

println("HASH TABLE LABORATORY WORK")
println("Variant: 12")
println("Hash function: first character code + last character code")
println("Collision resolution: chaining")
println()

// Читаем идентификаторы из файла.
val identifiers = try {
    File(fileName)
        .readLines()
        .map { it.trim() }
        .filter { it.isNotEmpty() }
} catch (e: Exception) {
    println("Error: file '$fileName' was not found.")
    throw e
```

```
}

// Добавляем идентификаторы в таблицу.
for (identifier in identifiers) {

    if (identifier.length > 32) {
        println(
            "Identifier '$identifier' was skipped: length is greater than 32."
        )
        continue
    }

    val inserted = hashTable.insert(identifier)

    if (!inserted) {
        println("Identifier '$identifier' was not added.")
    }
}

// Показываем таблицу.
hashTable.printTable()

// Статистика коллизий.
println()
println("STATISTICS")
println("=".repeat(60))

println("Number of identifiers: ${hashTable.getSize()}")
println("Total collisions: ${hashTable.getTotalCollisions()}")
println(
    "Average collisions: %.2f".format(
        hashTable.getAverageCollisions()
    )
)

println("=".repeat(60))

// Поиск идентификаторов.
println()
println("SEARCH")
println("Enter identifier to search.")
println("Enter 'exit' to finish.")
```

```
// Статистика сравнений только для запросов интерактивного поиска.
var searchCount = 0
var totalSearchComparisons = 0

while (true) {

    print("> ")

    val input = readlnOrNull()?.trim() ?: break

    if (input.equals("exit", ignoreCase = true)) {
        break
    }

    if (input.isEmpty()) {
        continue
    }

    val result = hashTable.search(input)
    searchCount++
    totalSearchComparisons += result.comparisons

    if (result.found) {
        println(
            "Identifier '$input' was found. " +
                    "Comparisons: ${result.comparisons}"
        )
    } else {
        println(
            "Identifier '$input' was not found. " +
                    "Comparisons: ${result.comparisons}"
        )
    }
}

// Среднее количество сравнений за один выполненный поиск.
val averageSearchComparisons = if (searchCount == 0) {
    0.0
} else {
    totalSearchComparisons.toDouble() / searchCount
}
```

```
println()
println("SEARCH STATISTICS")
println("=".repeat(60))
println("Number of searches: $searchCount")
println("Total comparisons: $totalSearchComparisons")
println("Average comparisons per search: %.2f".format(averageSearchComparisons))
println("=".repeat(60))

println()
println("Program finished.")
```

6 Результаты выполнения

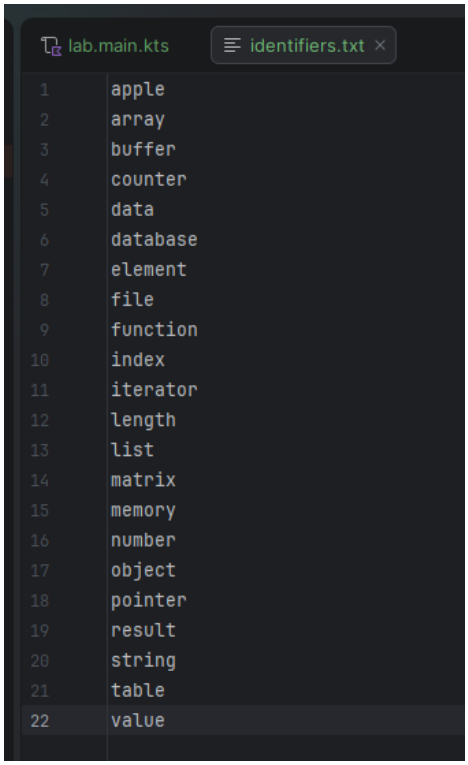


Рисунок 1 – Список слов для обработки

```
HASH TABLE LABORATORY WORK
Variant: 12
Hash function: first character code + last character code
Collision resolution: chaining


HASH TABLE
============================================================
Hash index | Chain
------------------------------------------------------------
         0 | empty
         1 | empty
         2 | [4: data]
         3 | [0: apple] -> [12: list] -> [15: number]
         4 | [2: buffer] -> [8: function] -> [9: index] -> [11: length]
         5 | [3: counter] -> [17: pointer]
         6 | [5: database] -> [16: object]
         7 | empty
         8 | [7: file] -> [13: matrix]
         9 | [6: element] -> [14: memory] -> [18: result] -> [20: table]
        10 | [1: array] -> [19: string]
        11 | [10: iterator] -> [21: value]
        12 | empty
------------------------------------------------------------
```

Рисунок 2 – Визуальный вывод данных в консоль

```
MAIN IDENTIFIER TABLE
============================================================
Index | Identifier | Next
------------------------------------------------------------
    0 | apple      |   12
    1 | array      |   19
    2 | buffer     |    8
    3 | counter    |   17
    4 | data       |   -1
    5 | database   |   16
    6 | element    |   14
    7 | file       |   13
    8 | function   |    9
    9 | index      |   11
   10 | iterator   |   21
   11 | length     |   -1
   12 | list       |   15
   13 | matrix     |   -1
   14 | memory     |   18
   15 | number     |   -1
   16 | object     |   -1
   17 | pointer    |   -1
   18 | result     |   20
   19 | string     |   -1
   20 | table      |   -1
   21 | value      |   -1
------------------------------------------------------------
```

Рисунок 3 – Таблица хранения данных

```
STATISTICS
============================================================
Number of identifiers: 22
Total collisions: 13
Average collisions: 0,59
============================================================

SEARCH
Enter identifier to search.
Enter 'exit' to finish.
> test
Identifier 'test' was not found. Comparisons: 2
> table
Identifier 'table' was found. Comparisons: 4
> index
Identifier 'index' was found. Comparisons: 3
> exit

SEARCH STATISTICS
============================================================
Number of searches: 3
Total comparisons: 9
Average comparisons per search: 3,00
============================================================
```

Рисунок 4 – Поиск слов и оценка сложности поиска

8 Выводы по проделанной работе

В ходе лабораторной работы была реализована хеш-таблица для хранения идентификаторов с использованием хеш-функции, основанной на сумме кодов первой и последней букв идентификатора.

Для разрешения коллизий был реализован метод цепочек. При возникновении коллизии идентификаторы с одинаковым хеш-значением объединяются в последовательную цепочку. Каждый элемент цепочки содержит ссылку на следующий элемент.

Также был реализован поиск идентификаторов по хеш-таблице. При поиске сначала вычисляется хеш искомого идентификатора, после чего просматривается соответствующая цепочка. Для каждого поиска подсчитывается количество выполненных сравнений.

В результате выполнения работы были изучены принципы построения хеш-таблиц, механизм возникновения коллизий и способ их разрешения с помощью метода цепочек. Метод цепочек позволяет эффективно организовать хранение элементов с одинаковыми хеш-значениями и не требует поиска свободной позиции при возникновении коллизии.